Compiling Array Languages for SIMD

V. E. McHale

January 27, 2025

Abstract

The C compilation model has dominated compilers literature but it opposes Single-Intruction, Multiple Data (SIMD). We articulate the assumptions behind textbook methods, where new CPUs diverge, and give a first cut of a better approach used in the Apple Array System. By reconsidering expectations about compiler optimization, we give a program for high-level languages to offer performant SIMD. Array languages turn out to be an especially good candidate, expressing efficient operations in a high-level language; we give examples in Apple.

1 Background

1.1 Textbook Compilation

C implementations compile the language's constructs (functions) to sequences of steps in assembly which read and write registers for arguments and return values defined by convention. The correspondence is elegant [1], but not suited to SIMD.

Consider a compiler written as mutually recursive functions:

```
writeExpr :: Expr -> Temp -> [Stmt]

writeFn :: Expr -> [Temp] -> Temp -> [Stmt]
```

The first function takes an expression and a register and creates a series of statements that will write that its value to the given register. The second takes an expression representing a function, registers to read arguments from, and a register to write the return value, and returns a series of statements. This is used in Appel [1][pp. 148-169], for instance.

Naïvely[2], we might try to compile `λas. map (λx. f x) as`, which maps over a vector, as follows:

[1] Kell [3] points out that C is uniquely suited to interfacing with other languages, in part because of this compilation practice.

[2] This was the approach taken in the Apple array system at first.

- Generate temporaries `x`, `y` for the argument and return value of `f`, respectively.
- Compile `λx. f x` using `writeFn`.
- Generate code to loop over the input vector's elements, reading from the input and putting the value in `x`, and then storing the result `y` in the output vector.

This is appealing because of its simplicity, and also has the advantage that `f` can be defined in another language. However in the case of

```
λas. map (λx. x*2) as
```

it would step over the input vector's elements one at a time. This is not as efficient as it could be—SIMD instructions can multiply several elements at once. Thus compiling `map` should not resort to a single canonical compiled version of `f`.

1.2 Functions in Theory

Landin [5] points out that Algol corresponds to the lambda calculus. Functions in the lambda calculus have a formal definition and are applicable to programming in practice, but only some have efficient SIMD implementations: `λx. 1-x^2` and `if x≥0 then x else 0` are both functions but the latter involves a conditional which in general is compiled to a jump; they must be treated differently during compilation but are not distinguished in the source language.

1.3 Limits of C

What makes C special in practice is that functions, which can be composed, are compiled to sequences of assembly instructions stored in object files; compositionality is borne through until linking. This compositionality is at odds with SIMD, where a high-level function `λx. x*2` should be compiled using different instructions depending on whether it is lifted to operate on an array. macOS provides

```
float expf(float x);
vFloat vexpf(vFloat x);
void vvexpf(float *y, const float *x, const int *n);
```

as three shared library functions to address this. A library providing a sigmoid function (for instance) would need to expose three versions in a shared library. The clumsiness proliferates and the mathematical entity no longer corresponds to compiled code but rather three versions of itself depending on the context in which it is used.

A compiler-as-a-library is appealing because it could dispatch different versions of a

2 Efficient Compilation

The full employment theorem for compiler writers states that it is impossible to write an algorithm that transforms an arbitrary program into an optimal version. Of course, such a "big hammer" would be handy, but the problem sitting before us is narrower. A compiler ferries a (possibly high-level) language to efficient code, and actually a source language with fewer low-level facilities offers more invariants. "Optimization" writ large gets much attention, but transformations that make high-level languages possible have not entered the computer science canon, particularly for functional or exotic languages.

"Amenability to efficient compilation" also informs the design of the source language. One can also consider languages where the programmer can expect efficient compilation by reasoning about the source-language in its own terms. This is not the case for C++, where efficiency of generated code can vary dramatically by compiler or across versions of the same compiler [6, 4, 2]. There is no contract with the user; performance depends on internals that are not documented, justified, analyzed with the same scrutiny as programming languages.

Array languages have a chance to do better since they naturally express computations that can be executed via SIMD, in a domain where such CPU features offer significant gains.

2.1 Typing & Indexing

```
float* matmul(float* a, float* b, int m, int n, int l) {
    float* c=malloc(m*l*sizeof(float));
    for (int i=0; i<m; i++) {
        for (int j=0; j<n; j++) {
            float z=0;
            for (int k=0; k<l; k++) {
                z+=A[i*m+k]*B[k*l+j];
            }
            C[i*m+j]=z;
        }
    }
    return c;
}
```

Suppose we wish to compute the 7-day moving average. We will do so using `` `\ ``, inspired by J's infix [7], which lifts a vector function to operate on windows of 7-vectors, arranging the (scalar) results into a vector. Examining type annotations, we see that the fold of the inner loop (`/`) is inferred have type `(float → float → float) → Vec 7 float → float`:

```
 > :ann λxs. [(+)/x%ℝ(:x)]\`7 xs
λ(xs : Vec (i + 7) float).
    ((\`7 : (Vec 7 float → float) → Vec (i + 7) float → Vec (i + 1) float)
```

```
    λ(x : Vec 7 float).
        ((% : float → float → float)
         ((/ : (float → float → float) → Vec 7 float → float)
          (+ : float → float → float)
          (x : Vec 7 float))
         ((ℝ : int → float) ((: : Vec 7 float → int) (x : Vec 7 float))))
    (xs : Vec (i + 7) float))
```

and in fact the inner loop is compiled without

```
> :asm λxs. [(+)/x%ℝ(:x)]\`7 xs

    ⋮
    ldr d2, [sp, x2, LSL #3]
    fadd d5, d5, d2
    add x4, x4, #0x1
apple_1:
    mov x2, #0x10
    add x2, x2, x4, LSL #3
    ldr q2, [sp, x2]
    fadd v3.2d, v3.2d, v2.2d
    add x4, x4, #0x2
    cmp x4, x3
    b.LT apple_1
    faddp d2, v3.2d
    fadd d5, d5, d2
    ⋮
```

Similar analyses are already performed by industrial compilers today, but with types, the information is present in the high-level language so that the programmer can expect

References

[1] Andrew Appel. *Modern Compiler Implementation in ML*. Cambridge University Press, 1998.

[2] hl3mukkel. Why does this code snippet produce radically different assembly code in c and c++? `https://stackoverflow.com/q/48837118/11296354`, 2018.

[3] Stephen Kell. Some were meant for c: the endurance of an unmanageable language. In *Proceedings of the 2017 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*, Onward! 2017, page 229–245, New York, NY, USA, 2017. Association for Computing Machinery.

[4] Marek Kolodziej. Matrix multiplication on cpu. https://marek.ai/matrix-multiplication-on-cpu.html, 2019.

[5] P. J. Landin. Correspondence between algol 60 and church's lambda-notation: part i. *Commun. ACM*, 8(2):89–101, February 1965.

[6] Dan Luu. Assembly v. intrinsics. https://danluu.com/assembly-intrinsics/, October 2014.

[7] Bob Therriault, Henry Rich, and Ian Clark. Vocabulary/bslash. https://code.jsoftware.com/wiki/Vocabulary/bslash, 2023.